would create the parameters `UserId=123` and `RecipeId=456`. *Structured logging* providers could store these values, in addition to the formatted log message `"User 123 loaded recipe 456"`. This makes it easier to search the logs for a particular `UserId` or `RecipeId`.

DEFINITION *Structured or semantic logging* attaches additional structure to log messages to make them more easily searchable and filterable. Rather than storing only text, it stores additional contextual information, typically as key-value pairs. JavaScript Object Notation (JSON) is a common format used for structured log messages.

Not all logging providers use semantic logging. The default console logging provider format doesn't, for example; the message is formatted to replace the placeholders, but there's no way of searching the console by key-value.

TIP You can enable JSON output for the console provider by calling `WebApplicationBuilder.Logging.AddJsonConsole()`. You can further customize the format of the provider, as described in the documentation at http://mng.bz/QP8v.

Even if you're not using structured logging initially, I recommend writing your log messages as though you are, with explicit placeholders and parameters. That way, if you decide to add a structured logging provider later, you'll immediately see the benefits. Additionally, I find that thinking about the parameters that you can log in this way prompts you to record more parameter values instead of only a log message. There's nothing more frustrating than seeing a message like `"Cannot insert record due to duplicate key"` but not having the key value logged!

TIP Generally speaking, I'm a fan of C#'s interpolated strings, but don't use them for your log messages when a placeholder and parameter would also